Taller Consumo de APIs

La compañía **CloudSpace Solutions** opera una plataforma web para la reserva y alquiler de espacios de trabajo (*coworking*) y salas de conferencias. Como parte de la modernización de su aplicación web (Laravel), la empresa requiere integrar un módulo comercial que permita a los usuarios realizar pagos en línea con tarjeta de crédito/débito y recibir notificaciones de confirmación por correo electrónico.

Para lograr esto, el equipo técnico ha estructurado el sistema alrededor de dos integraciones principales con servicios externos:

Procesamiento de Pagos (Stripe): La aplicación web debe ofrecer una interfaz de *checkout*. Cuando el usuario solicita realizar un pago, la petición debe procesarse mediante una capa de servicio dedicada que interactúa con la API de Stripe usando su SDK oficial en PHP para instanciar un intento de pago (*PaymentIntent*). De igual forma, el sistema debe escuchar y procesar eventos asíncronos vía *Webhook* enviados por la pasarela de pagos.

Notificaciones Transaccionales (SendGrid Mail): Tras la confirmación del pago, la aplicación debe procesar el envío de correos electrónicos utilizando la infraestructura de *SendGrid* a través de un servicio de envío desacoplado.

Como ingeniero *backend*, tu objetivo es implementar la aplicación web en Laravel, creando la capa de servicios (*Service Layer*), configurando las variables de entorno, la exención de tokens CSRF y las vistas Blade necesarias.

Requisitos Funcionales (RF)

Módulo 1: Procesamiento de Pagos (Stripe API)

RF 1.1 — Formulario Web de Checkout (GET /checkout): El sistema debe proveer una vista que con un formulario en el que los usuarios puedan ingresar el monto a pagar y el tipo de moneda.

RF 1.2 — Creación del PaymentIntent (POST /create-payment-intent): La petición del formulario debe procesarse mediante un controlador web. Este debe delegar la creación del cobro a la capa de servicio, específicamente a una clase para Stripe, la cual debe validar los datos e invocar la API de Stripe para retornar el *clientSecret* en formato JSON.

RF 1.3 — Renderizado y Confirmación en Cliente: La vista debe recibir el *clientSecret* generado e inicializar el componente de tarjeta con *Stripe Elements* para completar la transacción de forma segura.

Módulo 2: Recepción de Notificaciones Asíncronas (Stripe Webhook)

RF 2.1 — Recepción del Webhook (POST /stripe/webhook): El backend debe exponer una ruta para recibir las notificaciones enviadas por Stripe. En esta ruta debe registrar el éxito del pago en una base de datos relacional MySQL y responder con un código HTTP 200 OK.

Módulo 3: Envío de Correos Transaccionales (SendGrid Mail)

RF 3.1 — Servicio de Envío de Correo: El sistema debe integrar *SendGrid* utilizando una clase de servicio que consuma la API REST de SendGrid.

RF 3.2 — Módulo Web de Envío (GET /email, POST /email): El sistema debe ofrecer una vista con un formulario de datos de prueba para envío de emails y un controlador que procese el formulario y envíe correos a los destinatarios definidos.

Requisitos No Funcionales (RNF)

RNF 1 — Arquitectura y Abstracción de Servicios (Service Layer)

- La lógica de comunicación con las APIs externas no debe encontrarse dentro del cuerpo de controladores o de modelos *Eloquent*. Toda integración con terceros debe encapsularse en clases de servicio dedicadas. Los controladores deben limitarse exclusivamente a la gestión del flujo HTTP, validación de peticiones y retorno de respuestas y vistas.

RNF 2 — Seguridad y Variables de Entorno

- Todas las credenciales secretas (STRIPE_KEY, STRIPE_SECRET, SENDGRID_API_KEY, etc.) deben gestionarse en el archivo .env y mapearse en config/services.php.
- La ruta /stripe/webhook debe ser excluida expresamente de la validación de tokens CSRF en el middleware de la aplicación.

RNF 3 — Estándares Web y Manejo de Respuestas

- Las operaciones asíncronas deben retornar respuestas en formato JSON (200 OK, 400 Bad Request, 500 Internal Server Error, etc.), mientras que las interacciones web tradicionales deben retornar vistas Blade.

La entrega se debe realizar por medio de un repositorio público en GitHub enviado mediante la plataforma TEMA.